

Take-Home Assignment: Suspicious Photo Detection in Outlet Verification Images

Role: Sr. AI/ML Engineer

Context

Picture a field agent walking into a small mobile-recharge shop on a dusty street corner — snapping a few photos of the storefront, the signage, the counter — proof that the outlet is real, active, and compliant. Multiply that scene by thousands of outlets, visited again and again over months, and you get a sprawling photo archive meant to be the company's eyes on the ground.

Most of the time, the system works exactly as intended. But not every agent plays fair. Somewhere in that archive, a handful of workers are cutting corners — dropping in a photo of a random storefront, an old picture, or something that has nothing to do with the outlet at all, just to mark the visit as "done." Buried among thousands of legitimate images, these fakes are hard to catch by eye and impossible to catch by hand at scale.

That's the problem we need solved: given an outlet's accumulated photo history, can a system quietly flag the images that don't belong — without a human ever having to look at most of them?

Problem Statement

Given a dataset of outlet folders, where each folder contains multiple images captured across different visits for a single outlet (with no date/time metadata provided), build a system that identifies which image(s) in each folder are visually inconsistent with the outlet's overall photo history and should be flagged as suspicious.

Data Structure

```
dataset/  
outlet_0001/  
  img_01.jpg  
  img_02.jpg  
  img_03.jpg  
  ...  
outlet_0002/  
  img_01.jpg  
  ...
```

- Each folder = one outlet, aggregated across multiple visits; no capture date, timestamp, or visit order is available
- Images within a folder should be visually consistent with the outlet's identity, allowing for gradual, explainable change
- A small number of images per folder may be outliers (unrelated scenes, wrong locations, mismatched content)

Dataset

Dataset link: [dataset](#)

Expected Output Format

The system should produce a structured, per-outlet result – not just raw scores. Output can be delivered as JSON, CSV, or a DataFrame, but must follow this schema:

Per-outlet record:

Field	Type	Description
outlet_id	string	Identifier of the outlet folder
total_images	integer	Number of images evaluated in the folder
flagged_images	list	One entry per suspicious image (empty if none found)
ranking (optional)	list	All images in the folder ordered from most to least suspicious

Each entry inside flagged_images:

Field	Type	Description
file_name	string	Name of the flagged image
suspicion_score	float	Normalized score or confidence (e.g., 0-1) indicating how anomalous the image is
reason	string	Short, human-readable explanation of the flag (e.g., "low similarity to cluster centroid," "distinct background/signage")

Example:

```
{
  "outlet_id": "outlet_0001",
  "total_images": 6,
  "flagged_images": [
    {
      "file_name": "img_04.jpg",
      "suspicion_score": 0.87,
      "reason": "Distinct background and signage compared to the rest of the series"
    }
  ],
  "ranking": ["img_04.jpg", "img_02.jpg", "img_01.jpg", "img_03.jpg", "img_05.jpg", "img_06.jpg"]
}
```

The output should be generated for every outlet in the dataset, including outlets with no flagged images (returned with an empty `flagged_images` list rather than omitted).

Deliverables

Submit the following:

1. Working, runnable pipeline code
2. Similarity/scoring method used, with justification
3. Outlier detection rule used, with justification
4. Final results file (JSON/CSV) for the full dataset, following the output schema above
5. A short write-up (1 page max) covering rationale, trade-offs, scalability, and known limitations